课程设计报告书

题目：A Bottle Cap Pose Estimation Algorithm Based on Contour Analysis and Geometric Vectors

学　院 _____

专　业 _____

学生姓名 _____

学生学号 _____

学生班级 _____

指导教师 _____

课程编号 _____

课程学分 _____

起始日期 _____

教师评语	教师签名: 日期:
成绩评定	
备注	

Table of Content

1 Introduction to Research Background 4

2 Research Objectives.....4

3 Research Content 5

4 Research Design Plan 5

5 Presentation of Research Results.....9

6 Key Technologies and Innovations of the Research.....10

7 Conclusion and Outlook 11

8 Code 12

Reference 16

1. Introduction to Research Background

Beer, one of the most consumed beverages globally, accounts for annual sales exceeding 1.5 trillion units in United States. Of this total, merely 32% of the glass bottles are collected for recycling, with the remaining 68% being discarded or disposed of in landfills[1]. This discrepancy suggests a substantial potential for expansion within the beer bottle recycling sector.

Beer bottles are predominantly composed of glass, a material that requires an estimated one million years to fully decompose. The disposal of these bottles in landfills consumes substantial land resources and concurrently introduces significant pollutants into the environment. Consequently, the exploration of sustainable methods for beer bottle recycling is an urgent imperative[2].

The recycling process for waste beer bottles encompasses the following stages: collection, sorting, cleaning, and regeneration. Subsequent to the cleaning phase, all bottles must undergo an inspection to ascertain their suitability for continued use. Bottles that exhibit defects are considered unsuitable for recycling and must therefore be segregated and removed.

2. Research Objectives

This course aims to solve the problem of identifying non-conforming beer bottles in the recycling process through computer vision and image processing techniques. Specifically, the research tasks of this project can be divided into the following three parts:

1. For the given beer bottle images, detect the outer circular border of the cap and a small defect inside it.
2. Accurately calculate and obtain the center coordinates and radius of the two circles.
3. Compare the two images and calculate the rotation angle of the bottle cap based on the change in the relative positions of the inner and outer circle centers between the two images.

To accomplish these tasks, we set the following research goals and will solve them

one by one in the research process.

1. Develop an automatic program which can recognize the two circles of the bottle caps.
2. Verify the effectiveness and the robustness of our algorithm.
3. Provide a quantified result for users, here we can get accurate data for the center and the radius of two circles, which may be used in subsequent applications.

3. Research Content

This research primarily consists of the following three core modules.

1. Study how to use a series of preprocessing operations to eliminate redundant information and noise from the images, enhancing the target features to create ideal conditions for subsequent feature extraction.
2. Research and implement a feature detection algorithm. The core content includes how to stably identify the main contour of the bottle cap using the geometric properties of contours, and how to design a filtering logic with multiple dimensions to accurately locate the target inner circle from numerous interfering contours.
3. Consider how to translate the detected image features into a computable mathematical model. This is achieved by constructing a pose vector that represents the orientation of the cap and comparing these vectors from the two images to accurately calculate the rotation angle.

4. Research Design Plan

Based on a preliminary visual analysis of the beer bottle cap images, its structure is observed to comprise three distinct circular features: an outer, large circle with a serrated perimeter; an intermediate, slightly smaller circle within it; and a small, inner circle positioned eccentrically within the intermediate one. This observation allows for the subsequent research to be delineated into the following steps.

1. Inner and Outer Circles Detection
    1) Read and Binarization

        First, the program reads the image from the specified path and directly

converts it to a grayscale image. This is done to simplify the image information, as we are concerned with shapes and contours, not colors.

Next, the grayscale image undergoes binarization. All pixel values above 127 become 255 (white), and those below 127 become 0 (black). This way, the bright parts of the bottle cap (the features we are interested in) become the white foreground, while the dark background becomes black.

2) Close Operation

Defines a kernel, which is a small shape matrix used for subsequent image processing. Here, `MORPH_ELLIPSE` (an ellipse) of size 13x13 is used.

Executes a morphological closing operation. This operation effectively fills small black holes within the foreground object (white areas) and connects broken white regions, making the bottle cap's contour smoother and more complete for subsequent detection.

3) Contours Detection

Finds the contours of all closed shapes in the processed binary image. `contours` is a list containing the point sets of all contours

4) Find Outer and Inner Circles

A. Find Outer Circle

The outer circle is the largest circle in the image, so we can use `max` and `cv2.contourArea` to find contour with the largest area.

Performs a minimum enclosing circle fit on the convex hull of the largest contour. Using the convex hull results in a smoother, more regular circle, allowing for the precise calculation of the outer circle's center `(outer_cx, outer_cy)` and radius `outer_radius`.

Then, the detected outer circle is drawn in red.

B. Find Inner Circle

We use a loop to iterate through all other contours except the largest one, filtering for our target inner circle using a series of strict conditions.

The area must be smaller than the outer circle's but not too small, to filter out noise. When a circle was discovered, a minimum enclosing circle is fitted to the current contour, and the radius of this circle must be within a predefined reasonable range (8 to 18 pixels in our code).

After thar, we will calculate the distance between this circle's center and the outer circle's center. The distance from this circle's center to the outer circle's edge is also a important value to be calculated.

The most critical filtering logic is that the inner circle to be a certain distance away from the outer circle's center (the `center_dist` condition, which excludes interference from the cap's very center) and that the inner circle itself is not too close to the outer circle's edge (the `edge_dist` condition).

Once the first contour that fully satisfied all conditions, it is considered the target inner circle, and the search stops.

Then, the detected inner circle is drawn in green.

5) Save

The resulting image with the red and green circles drawn on it is saved locally, and the detected inner and outer circle data (center coordinates and radius) are returned.

2. Calculate Angle Difference

First, we construct a vector for each image, originating from the center of the outer circle and pointing toward the center of the inner circle. These vectors are then normalized to unit vectors while preserving their directions.

Next, we compute the dot product of the two unit vectors. According to linear algebra, the dot product of two normalized vectors equals the cosine of

the angle between them.

To obtain the angle in radians, we apply the `arccos` function to the dot product. To avoid numerical errors that may cause the dot product to slightly exceed the range [−1,1], we use `np.clip` to constrain its value within this valid interval.

Finally, the angle in radians is converted into degrees and returned as the result.

The following images show how the algorithm works to detect the circles in the two given images.

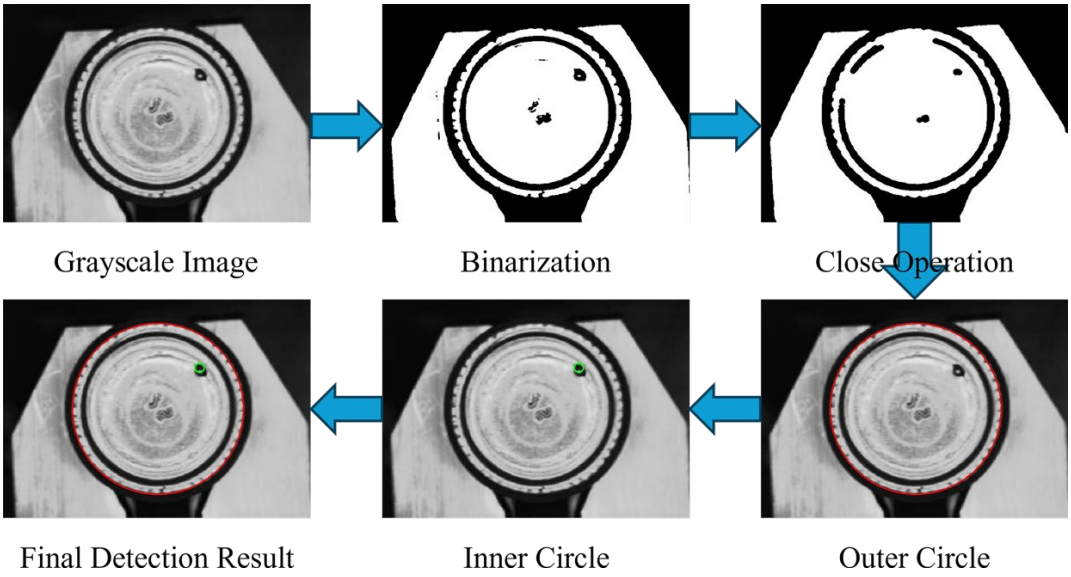

Figure 1: The workflow of image 1

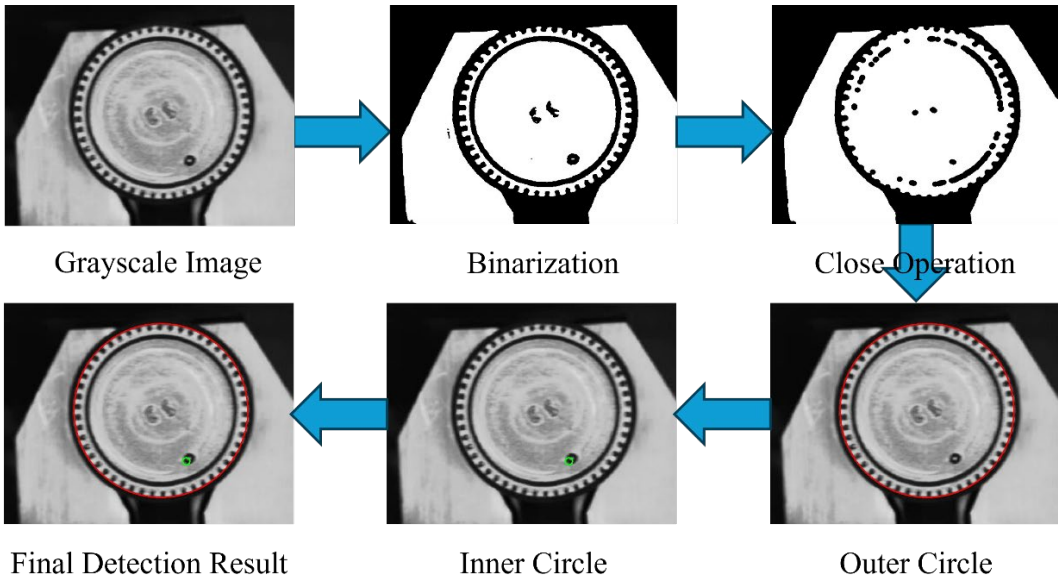

Figure 2: The workflow of image 2

5. Presentation of Research Results

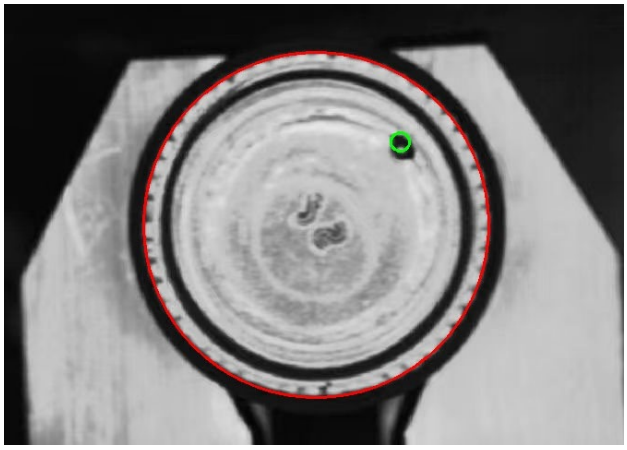

Figure 3: The detection result of image 1

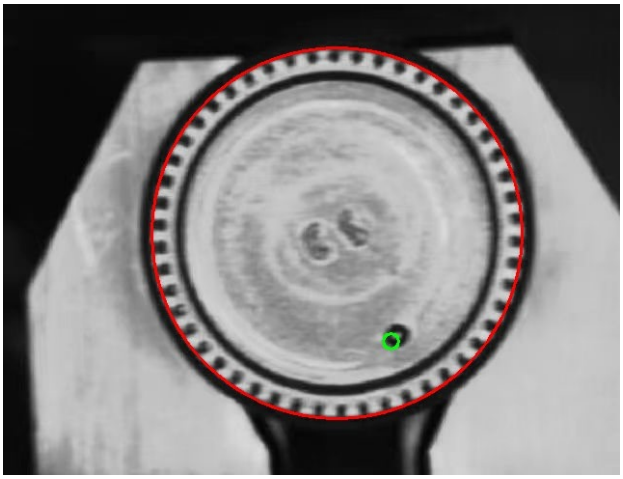

Figure 4: The detection result of image 2

It is clear to see that both the outer circle and inner circle are detected accurately in the red and green color. And the output text of the program is shown following.

```
Image 1 - Outer circle: ((338.7787170410156, 239.35293579101562), 187.38504028320312)
Image 1 - Inner circle: ((429.5, 149.0), 10.920265197753906)
Image 2 - Outer circle: ((346.4438171386719, 237.4427032470703), 192.87985229492188)
Image 2 - Inner circle: ((402.5, 349.5), 8.860122680664062)
Angle difference: 108.31 degrees
```

Figure 5: Output text

The output text includes the center position of the outer circle and the inner circle. The angle difference is also given in the output. We first used the software *Adobe Photograph* to scale the two images such that the diameters of the large (outer) circles

matched, ensuring that no distortion was introduced during the process. At this point, the rotation angle between the two images was manually measured to be 101.5 degrees. This value was assumed to be the reference (or ground truth) rotation angle. Our program produced a result of 108.31 degrees, yielding a relative error of

$$\frac{108.31 - 101.5}{101.5} = 0.067$$

We consider this error to be an acceptable value.

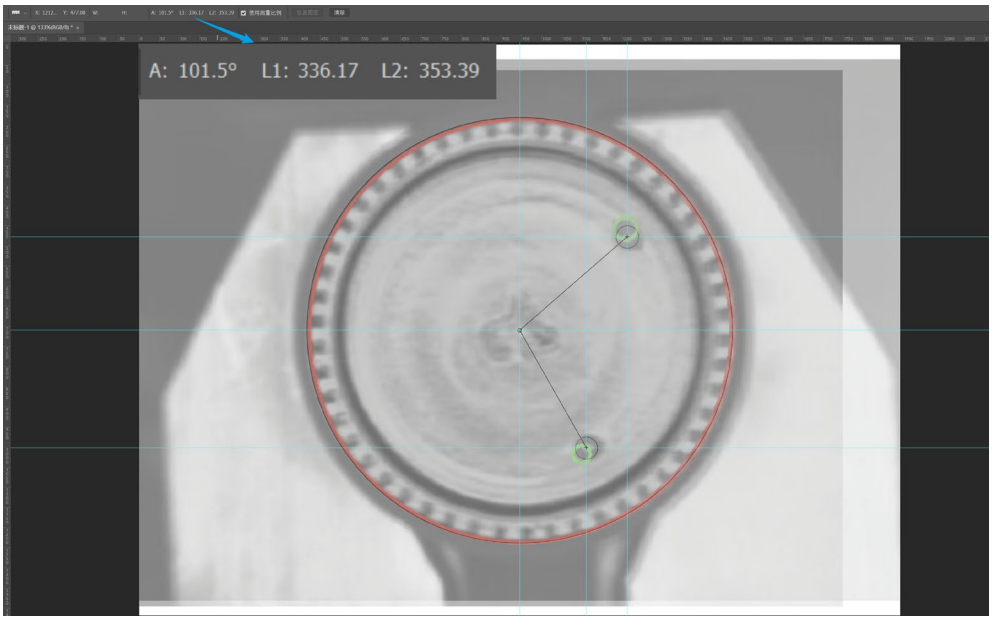

Figure 6: Manually measurement of the rotation angle

6. Key Technologies and Innovations of the Research

The innovation of this research lies not in inventing a completely new algorithm, but in the intelligent combination and optimization of existing, mature technologies to form an efficient and precise solution for a specific application problem. Our research mainly consists of three key technologies.

1. Detection Strategy via Contour Analysis

Compared to the Hough Circle Transform[3] (we have tried this method but the performance is not good), which is highly sensitive to parameters and the perfection of the circle's shape, this design chooses a more robust technical path. We adopt a strategy of "first identify the object's contour, then perform a geometric fit on that contour." This method has a higher tolerance for imperfect circles caused by lighting, wear, or other factors, allowing for more stable localization of the main bottle cap. This forms the

foundation for the success of the entire solution.

2. Precise Target Filtering Logic with Multi-Dimensional Constraints

This is the core innovation of this solution. To accurately find the target inner circle among numerous interferences (while excluding the bright spot in the center), we did not rely on a single feature. Instead, we designed a comprehensive, multi-dimensional filtering logic. It simultaneously considers four geometric constraints: the contour's area (to exclude noise), the fitted circle's radius (to limit size), the relative distance from the outer circle's center (to exclude central interference), and the distance from the outer circle's edge (to ensure a reasonable position). This "filter" of combined conditions uniquely and accurately locks onto the target, greatly enhancing the precision of the detection.

3. Geometric Decoupling Measurement via Pose Vector

We simplified a pose estimation problem into a vector calculation problem through geometric modeling. By constructing a "pose vector" pointing from the outer circle's center to the inner circle's center, we successfully decoupled the cap's rotational motion from its translational motion within the image. The final angle calculation depends only on the change in this vector's direction, and is not affected by the cap's different positions in the two images. This approach is conceptually clear and fundamentally ensures the accuracy of the rotation angle calculation.

7. Conclusion and Outlook

This research has successfully designed and implemented an automated visual inspection system using Python and OpenCV. The system is capable of accurately identifying the inner and outer circular features on beer bottle cap images and precisely calculating the cap's rotation angle based on these features. Through a pipeline of image preprocessing, contour-based feature extraction with multi-constraint filtering, and geometric decoupling measurement via vector algebra, this solution has fully achieved the preset research objectives.

The results demonstrate that by judiciously combining and optimizing a series of classic computer vision techniques, specific industrial automation inspection problems can be solved efficiently and stably. This project not only validates the feasibility and accuracy of the chosen technical approach but also provides a valuable reference for tackling similar geometry-based pose estimation problems.

However, there is still room for optimization from the perspective of broad industrial applicability and robustness. The current solution uses a fixed global threshold for binarization, which is sensitive to changes in lighting. In the future, Adaptive Thresholding algorithms could be employed. This solution also relies on well-defined circular features. If the bottle cap pattern changes to an irregular shape or a complex logo, this method would fail. Future work could explore the use of feature point matching methods. These methods find unique keypoints in an image that are invariant to rotation and scale and can calculate an object's rotation and translation by matching these point sets, making them much more versatile. For the ultimate challenges involving highly diverse models, extremely complex backgrounds, or severe occlusions, deep learning solutions can be explored. While this would require significant annotated data and training resources, its potential in generalization and handling complex scenes is unparalleled by traditional methods.

8. Code

Here is the entire code for this research.

```
import cv2

import numpy as np

import math

def detect_circles(image_path,
                                    image_index,
```

```
                    min_radius=8,
                    max_radius=18,
                    min_distance_to_center_ratio=0.3,
                    min_distance_to_edge=50):
    # Read and binarization
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    cv2.imwrite(f"imgs/{image_index}_1_gray.jpg", img)

    _, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
    cv2.imwrite(f"imgs/{image_index}_2_binary.jpg", binary)

    # Close operation to remove jagged edges and fill gaps
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (13, 13))
    closed = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)
    cv2.imwrite(f"imgs/{image_index}_3_closed.jpg", closed)

    # Find contours
    contours, _ = cv2.findContours(closed, cv2.RETR_TREE,
cv2.CHAIN_APPROX_SIMPLE)

    # Prepare images
    base_img = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
    inner_only_img = base_img.copy()    # for inner circle only
    final_img = base_img.copy()         # for both circles
```

```
    # Find outer circle
    largest_contour = max(contours, key=cv2.contourArea)
    (outer_cx, outer_cy), outer_radius = cv2.minEnclosingCircle(cv2.convexHull(largest_contour))
    outer_area = cv2.contourArea(largest_contour)

    # Draw outer circle
    cv2.circle(final_img, (int(outer_cx), int(outer_cy)), int(outer_radius), (0, 0, 255), 2)    # red outer
    cv2.imwrite(f"imgs/{image_index}_4_outer_circle.jpg", final_img)

    # Find inner circle
    inner_circle = None
    for cnt in contours:
        area = cv2.contourArea(cnt)
        if area < outer_area and area > 50:
            (x, y), radius = cv2.minEnclosingCircle(cnt)
            if min_radius < radius < max_radius:
                center_dist = np.sqrt((x - outer_cx) ** 2 + (y - outer_cy) ** 2)
                edge_dist = abs(center_dist - outer_radius)

```

```
                if center_dist > outer_radius * min_distance_to_center_ratio and edge_dist > min_distance_to_edge:
                    inner_circle = ((x, y), radius)
                    break

    # Draw inner circle only
    if inner_circle:
        (x, y), radius = inner_circle
        cv2.circle(inner_only_img, (int(x), int(y)), int(radius), (0, 255, 0),
        cv2.imwrite(f"imgs/{image_index}_5_inner_circle.jpg", inner_only_img)

    # Draw both circles on final image
    cv2.circle(final_img, (int(outer_cx), int(outer_cy)), int(outer_radius), (0, 0, 255), 2)  # red outer
    if inner_circle:
        cv2.circle(final_img, (int(x), int(y)), int(radius), (0, 255, 0), 2)  # green inner

    # Save final result
    output_path = f"imgs/{image_index}_6_result.jpg"
    cv2.imwrite(output_path, final_img)
    output_path = image_path.replace('.jpg', '_result.jpg')
    cv2.imwrite(output_path, final_img)
```

```

    outer_circle = ((outer_cx, outer_cy), outer_radius)
    return outer_circle, inner_circle

def calculate_angle_diff(vec1, vec2):
    unit_vec1 = vec1 / np.linalg.norm(vec1)
    unit_vec2 = vec2 / np.linalg.norm(vec2)
    dot_product = np.dot(unit_vec1, unit_vec2)
    angle_rad = np.arccos(np.clip(dot_product, -1.0, 1.0))
    return math.degrees(angle_rad)

outer1, inner1 = detect_circles("beerbottle1.jpg", image_index=1)
# print("Image 1 - Outer circle:", outer1)
# print("Image 1 - Inner circle:", inner1)

outer2, inner2 = detect_circles("beerbottle2.jpg", image_index=2)
# print("Image 2 - Outer circle:", outer2)
# print("Image 2 - Inner circle:", inner2)

# Calculate angle difference
if inner1 and inner2:
    vec1 = np.array([inner1[0][0] - outer1[0][0], inner1[0][1] - outer1[0][1]])
```

```
        vec2 = np.array([inner2[0][0] - outer2[0][0], inner2[0][1] - outer2[0][1]])

        angle_diff = calculate_angle_diff(vec1, vec2)
        print(f"Angle difference: {angle_diff:.2f} degrees")
    else:
        print("Cannot calculate angle difference - Circles not found in both images")
```

Reference

[1]. Millette, S., & Morawski, C. (2025, February 24). Unlocking recycling potential. *Resource Recycling*. Retrieved from Resource Recycling. https://resource-recycling.com/recycling/2025/02/24/unlocking-recycling-potential/

[2]. Wikipedia contributors. Container-deposit legislation. In Wikipedia, The Free Encyclopedia. Retrieved June 10, 2025. https://en.wikipedia.org/wiki/Container-deposit_legislation

[3]. PyImageSearch. (2014, July 21). Detecting Circles in Images using OpenCV and Hough Circles. Retrieved from PyImageSearch. https://pyimagesearch.com/2014/07/21/detecting-circles-images-using-opencv-hough-circles/